2014 12th Symposium on Neural Network Applications in Electrical Engineering (NEUREL)
School of Electrical Engineering, University of Belgrade, Serbia, November, 25-27, 2014

Implementation of a Feed-forward Artificial Neural Network in VHDL on FPGA

Philippe Dondon, Julien Carvalho, Rémi Gardere, Paul Lahalle, Georgi Tsenov and Valeri Mladenov

***Abstract*— Describing an Artificial Neural Network (ANN) using VHDL allows a further implementation of such a system on FPGA. Indeed, the principal point of using FPGA for ANNs is flexibility that gives it an advantage toward other systems like ASICS which are entirely dedicated to one unique architecture and allowance to parallel programming, which is inherent to ANN calculation system and one of their advantages. Usually FPGAs do not have unlimited logical resources integrated in a single package and this limitation forcesrequirement for optimizations for the design in order to have the best efficiency in terms of speed and resource consumption. This paper deals with the VHDL designing problems which can be encountered when trying to describe and implement such ANNs on FPGAs.**

***Index Terms*— FPGA implementation, neural networks, nonlinear systems, VHDL.**

I. Introduction

INSPIRED by brain functioning, artificial neural networks are usually implemented either for forecasting or for classification tasks. This is why they are frequently used for medical diagnoses, target detection for military applications, and financial forecasting. ANNs are chosen for their flexibility which allows them to process various data typeswith high speed, thanks to the network parallelism.

Considering these particularities of ANNs, FPGAs represent a very interesting solution for a hardware implementation since they are reconfigurable. Thus, in opposition with VLSI (Very Large Scale Integration) circuits, modifying the characteristics or the structure of an ANN is possible without a rebuilt of the circuit, even if the performances are lower using FPGAs. Moreover parallelism is inherent to FPGAs and they have short design cycles.

Considering the relevance of choosing FPGA in order to implement ANNs, the purpose of our work is to describe in VHDL an ANN dedicated to be implemented on a Zynq-7000 EPP 7Z020 Zedboard Kit.

We decided to select a simple Feedforward Neural Network dedicated to pattern recognition and composed of one hidden layer with three neurons and one neuron in the output layer.

II. Learning Process, Data Format and Memories

In an ANN, for each neuron the strength of synaptic connections is represented by weights attributed to each input

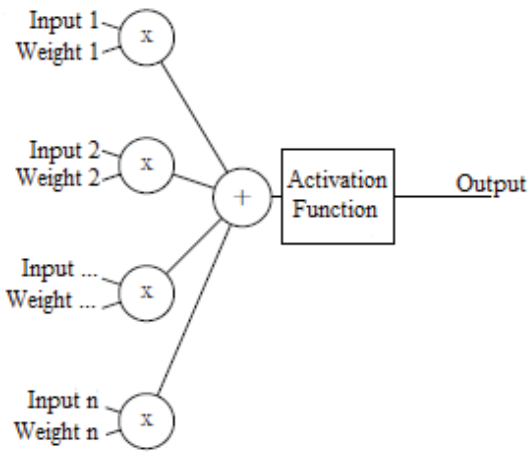

Fig. 1. Artificial Neuron model.

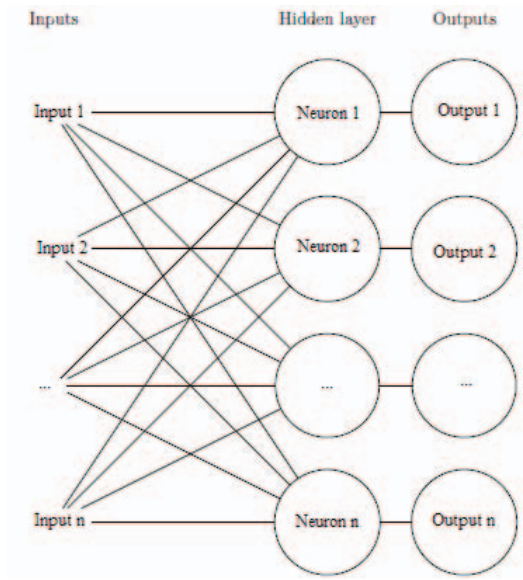

Fig. 2. Example of ANN.

Philippe Dondon is with the ENSEIRB-MATMECA, École Nationale Supérieure d'Électronique(e-mail: Philippe.Dondon@enseirb-matmeca.fr)
Julien Carvalho is with the ENSEIRB-MATMECA, École Nationale Supérieure d'Électronique
Rémi Gardere is with the ENSEIRB-MATMECA, École Nationale Supérieure d'Électronique
Paul Lahalle is with the ENSEIRB-MATMECA, École Nationale Supérieure d'Électronique
Georgi Tsenov is with the Department of Electrical engineering of the Faculty of Automation of Technical University of-Sofia
Valeri Mladenov is with the Department of Electrical engineering of the Faculty of Automation of Technical University of-Sofia (e-mail:valerim@tu-sofia.bg).

978-1-4799-5888-7/14/$31.00 ©2014 IEEE

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:03:26 UTC from IEEE Xplore. Restrictions apply.

(and a bias input, which is attributed to a constant input of 1). These weights must be set at certain values in order to be able to allow the successful recognition of the selected patterns. This is done through an iterative optimization procedure named the learning/training process of ANN.

One of the problems encountered when dealing with FPGAs for ANNs implementation is the size of the networks that reduces the possibilities for implementation. That is why we decided since the learning process is done prior to the utilization of the neural network to do the trainingstage with MATLAB, using the Neural Network Toolbox to train the neural network weights and obtain the trained weights in matrix form.

A. Data Format

The use of real numbers in some calculations forces the designer to define a new type. Each real number is defined on a 32 bits vector: 1 for the sign (the higher one), 21 for the integer part and 10 for the fractional part. With this chosen code 1 is equal to 1024, 0.5 is equal to 512. With this code integer number can reach $\frac{2^{32}}{2^{10}} = 4194304$ by keeping a good accuracy with both fractional and integer parts.

TABLE I
REAL NUMBER SHAPE

s : 1 bit	Int : 21 bits	Frac : 10 bits

B. Memories

Memories enable to stock data. In this particular PFGA implementation, only RAMs and ROMs were used. ROMs (Read Only Memory) stock data which can only be read, so we chose to implement two ROMs, one to stock the weight and one to stock the network parameters.

In a RAM (Random Access Memory) data can be updated and read, so we used one to stock the inputs of the networks that will be written directly through a test file and one for the outputs of each hidden layers that will be modified by the neurons themselves and read by the ones of the next layer.

In order to enable an easy updating of the data, memory blocks were added as parts of a network component and not as part of neurons components. An arrays that are included in neurons blocks could have been designed, but this implementation would have had caused problems during the updating of data because instead of modifying only ROMs blocks, each neuron should have been modified also.

III. Neurons Implementation and Implementation of the Network

We decided to describe each neuron as a top level whose components are the calculation and treatment units.

Calculation unit

The calculation unit contains the activation function selection and calculation processes. This module communicates only with the treatment unit which sends it 3 signals: Select_function (value previously gotten in the parameters ROM), data_in (result of the ponderate sum) and calc (sigmoid transfer function launching signal). In exchange, the calculation units send the result of the calculated activation function across value_heav, value_sig or value_lin, that correspond to the three activation functions available : Heaviside, Sigmoid and linear.

For the sigmoid function, a MATLAB study was done with the “logsig” function which takes real values from 0 to 1. Then this function is sampled from -5 to 5 with different levels of sampling: a step of 1 for the first level (11 samples), 0.5 for the second one (21 samples), until 0.01 for the fourth and last level (1001 samples). Eventually, all the values below -5 equal 0 and above 5 equal 1. It appears that the third and fourth levels (101 samples) are the minima levels needed to get a similarity between the real functions sampled and the theory. However, we were forced to use the last level of sampling because of the lack of precision of the third level and to reduce errors.

When implementing in VHDL, two arrays were created. The first one contains the value of the activation function step Y-axis values. A total of 100 steps were chosen so that the array has 100 memory boxes. The other one contains the X-axis limits values for each step. In order to find out the good interval, a state machine was implemented. This state machine will start by verifying the sign of the result sent by the treatment unit before determining which is the corresponding value in the Y-axis array.

Treatment unit

This state machine manages the actions between all modules. First of all it gets data from the network parameters ROM: the threshold, the inputs and the outputs numbers, and the activation function chosen. Then, it gets all the weights by packets of 32 bits from the weights ROM and keeps them into a concatenated shape then the initialization of the neuron is done. When a neuron has finished initializing himself, it has all the data needed and is ready to calculate its output.

The first step of the calculation is the updating of the network inputs RAM by the first neuron of the network, after what the ponderate sum can be calculated by each neuron and the result written in the RAMs.

We have studied two different ways of implementing the network using the previous Neuron modules, but the global structure remains the same:

One module per neuron

This choice consists in using one block for each neuron. In this case, the neurons initialize themselves one by one and wait for the others at each step. When they are all initialized, they calculate the ponderate sum simultaneously and when it is done, write the result in the RAMs.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:03:26 UTC from IEEE Xplore. Restrictions apply.

This solution reachesquite an interesting speed since the calculation processes are done in parallel and allows doing the initialization part only once but it uses a lot of the FPGA resources.

One module per layer

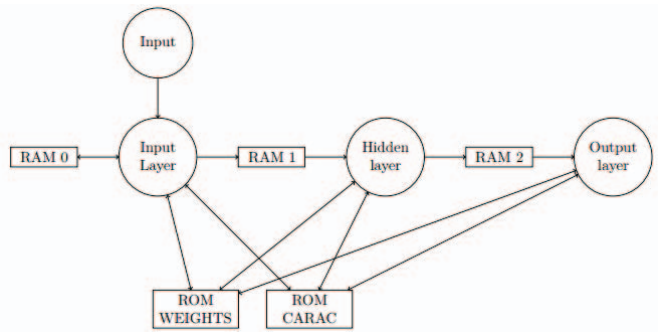

Fig. 3. Single-hidden-layer network structure.

In this solution we only implement one neuron per layer and we add an additional module for each layer to that will loopback on the neuron module every time a result is given, providing specifications for the neuron about his position in the network. In this situation the neuron never has to wait for the other neurons of the network during his initialization and treatment once it is activated.

This way the project includes less components than in the first choice so less resources are consumed on the FPGA. Nevertheless, in this case, the neurons of the network are working one by one and their initialization must be done again when new inputs are processed.

IV. Results

The training and testing of the Neural Networks process was done in MATLAB. In the FPGA implementation directly the weight values were flashed and the system. With only testing of the FPGA neural network model we observed the same results as the MATLAB model. Since for Neural Networks the main compute requirements are in the training phase it makes sense to have the training done on PC and to have the trained neural network converted into hardware implementation.

The minimal clock period for operationof every neuron in discrete time intervals is is about 2.355 ns. This period depends on the FPGA clock frequency and on the implementation of each module. Considering the minimal period of each module, it seems that the Core module imposes its minimal period to the rest of the system. It is possible to improve the speed characteristics by taking a faster clock or by optimizing the VHDL implementation of the slowest module.

```
Minimum period: 2.388ns (Maximum Frequency: 418.761MHz)
Minimum input arrival time before clock: 2.470ns
Maximum output required time after clock: 0.788ns
Maximum combinational path delay: No path found
```

Fig. 4. Network operating frequency.

In this respect, the faster a neuron module is clocked, the faster the network can process information. The measured minimal period equals 2.388 ns namely the one of one neuron.

TABLE II
RESOURCES USED BY A NEURAL NETWORK OF 3 INPUTS, 3 HIDDEN NEURONS ON 1 LAYER AND 1 OUTPUT WITH THE FIRST SOLUTION

Device Utilization Summary (estimated values)			[-]
Logic Utilization	**Used**	**Available**	**Utilization**
Number of Slice Registers	4632	106400	4%
Number of Slice LUTs	7739	53200	14%
Number of fully used LUT-FF pairs	4470	7901	56%
Number of bonded IOBs	99	200	49%
Number of BUFG/BUFGCTRLs	19	32	59%
Number of DSP48E1s	16	220	7%

Considering the resources we use, the number of DSP enables the implementation of a maximum of 55 neurons (220/4). This limitation can be even lower seeing used LUTs but Xilinx optimization process doesn't enable to forecast the number of LUTs which will be used before the synthesis.
A second study case is shown on Table III.

TABLE III
RESOURCES USED BY A NEURAL NETWORK OF 3 INPUTS, 3 HIDDEN NEURONS ON 1 LAYER AND 1 OUTPUT WITH THE SECOND SOLUTION

Device Utilization Summary (estimated values)			[-]
Logic Utilization	**Used**	**Available**	**Utilization**
Number of Slice Registers	2306	106400	2%
Number of Slice LUTs	3789	53200	7%
Number of fully used LUT-FF pairs	2221	3874	57%
Number of bonded IOBs	131	200	65%
Number of BUFG/BUFGCTRLs	16	32	50%
Number of DSP48E1s	8	220	3%

In this choice, the number of DSP and LUTs is really lower than the previous choice, but it is difficult to evaluate its evolution because it is not linear following the number of neurons like in the first solution but depends on the structures of the network (numbers of layers and neurons per layer). However we can see that it offers the possibility of implementing wider networks but that will process the data more slowly.

We did not have had access to other FPGA board rather than the Xilinx Zynq-7000 EPP 7Z020 Zedboard Kit, so comparison in terms of execution speed and capacity with other FPGA kits like Raspberry Pi or Xilinx Spartan or Virtex series is not performed in this paper.

V. Conclusion

It has been proved that using FPGAs to implement ANN is a good option since VHDL offers many design possibilities and advantages when, at the same time, FPGA bring their flexibility and cost-efficiency.

This work could still be improved, for instance the transfer of data from MATLAB to VHDL design tool could be done by programming an automatic data converter (for example a Simulink file).

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:03:26 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Haykin, "Neural Networks: a comprehensive foundation", 2nd Edition, Prentice Hall, 1999

[2] S. Haykin, "*Neural Networks*", Macmillan College Publishing Co. Inc. 1994.

[3] The MathWorks, Inc., MATLAB 2010b, 24 Prime Park Way, Natick MA, 2010.

[4] H.Demuth, M.Beale, M.Hagan, Neural Network Toolbox 5, User's Guide, The Mathworks, 2007.

[5] M.T.Hagan, H.B.Demuth, M.H.Beale, Neural Network Design, Boston, MA: PWS Publishing, 1996.

[6] D.Zhang, Parallel VLSI Neural System Design, Springer-Verlag, Singapore, 1999.

[7] D. W. Tank and J. J. Hopfield. "Simple "neural" optimization networks: an A/D converter, signal decision circuit, and a linear programming circuit", IEEE Trans. Circuits and Systems, CAS-33, pp.533-541, 1986.

[8] L. O. Chua and G. N. Lin. "Non-linear programming without computation", IEEE Trans. Circuits and Systems, CAS-31, pp.182-186, 1984.

[9] A.J.Agranat, C.F.Neugebauer, R.D.Nelson and A.Yariv, "The CCD neural processor: A neural network integrated circuit with 65536 programmable analog synapses", *IEEE Trans. Circuits Syst*., vol.37, pp.1073-1075, Aug.1990.

[10] A.J.Agranat, C.F.Neugebauer, R.D.Nelson and A.Yariv, "The CCD neural processor: A neural network integrated circuit with 65536 programmable analog synapses", *IEEE Trans. Circuits Syst*., vol.37, pp.1073-1075, Aug.1990.

[11] C.G.Y.Lau and B.Widrow, Eds. "Special Issue on Neural Networks", Proceedings IEEE, Vol.78, Sept./Oct.1990.

Authorized licensed use limited to: Cyprus University of Technology. Downloaded on June 04,2025 at 12:03:26 UTC from IEEE Xplore. Restrictions apply.